

Explain the chapter <Dictionary> in detail.

Break down all concepts step by step in very simple language.

Include:

- Clear definitions of each concept
- Important formulas with parameter names explained
- Real-world examples for each concept
- Visualizations or diagrams to make it intuitive (use graphs and colorful notes)
- Conceptual code examples (Python preferred) with dry run tables
- Common mistakes and how to avoid them
- Interview-style questions with answers for practice

Make the explanation very conceptual, beginner-friendly, and structured like a study guide.

Use graphs, colorful notes, and highlight formulas with parameter names for clarity.

Provide complete Python code examples with dry run tables for understanding.

colorful notes and diagram and difficult word explain to easy meaning explain all concept through diagram and color full Notes basics to advanced code and write every word very easy mean

Python Dictionary — Complete Beginner-to-Advanced Study Guide

 **Big idea:** A Python **dictionary** stores data in **key : value pairs**.

Think of it like a real dictionary: you look up a **word (key)** to find its **meaning (value)**.

1. What is a Dictionary?

Simple definition

A **dictionary** is a Python data structure that stores information as:

KEY → VALUE

Example:

```
student = {  
    "name": "Aamir",  
    "age": 22,  
}
```

</> Python

Run

Here:

Key	Value
"name"	"Aamir"
"age"	22
"course"	"Python"

 Easy meaning of difficult words

Word	Easy meaning
Dictionary	A collection of key-value pairs
Key	The name/label used to find data
Value	The actual data stored
Pair	Two things connected together
Mutable	Can be changed
Unique	Cannot be repeated
Lookup	Finding something
Hashable	A type that Python can safely use as a dictionary key

2. Real-World Example

Imagine a student ID card:

Student ID → A101
Name → Aamir
Age → 22
Course → Python



Python dictionary:

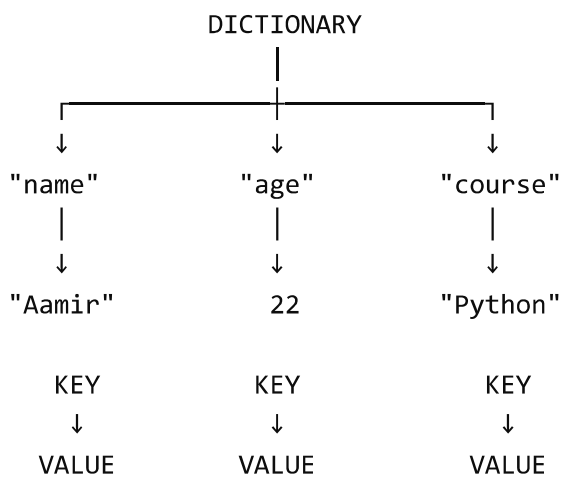
```
student = {  
    "student_id": "A101",  
    "name": "Aamir",  
    "age": 22,  
}
```

 Python



 Run

Visual picture



Remember:

Dictionary = **Label** → **Information**

3. Creating a Dictionary

Use curly brackets:

```
student = {  
    "name": "Aamir",
```

</> Python



Run

Syntax:

```
dictionary = {  
    key1: value1,  
    key2: value2,
```

</> Python



Run

Example

```
car = {  
    "brand": "Toyota",  
    "model": "Camry",  
    "year": 2025  
}
```

</> Python



Run

Output:

```
{'brand': 'Toyota', 'model': 'Camry', 'year': 2025}
```



COLOR NOTE — Dictionary Structure

PYTHON DICTIONARY		
KEY	:	VALUE
"name"	:	"Aamir"
"age"	:	22
"city"	:	"Delhi"



The colon `:` connects the key and value.

```
"name": "Aamir"
```

`</>` Python



▶ Run

4. Dictionary vs List

This is very important.

List

A list normally uses a **position/index**.

```
students = ["Aamir", "Rahul", "John"]
```

`</>` Python



▶ Run

Output:

Aamir



You say:

Give me item at position 0



Dictionary

A dictionary uses a **key**.

```
student = {  
    "name": "Aamir",  
    "age": 22  
}
```

`</>` Python



▶ Run

Output:

Aamir



You say:

Give me the value associated with "name"



Visual comparison

LIST



Index		Value
0	→	Aamir
1	→	Rahul
2	→	John

DICTIONARY

Key		Value
"name"	→	Aamir
"age"	→	22

🧠 Easy memory trick:

List → **position**

Dictionary → **name/key**

5. Dictionary Keys

A dictionary key identifies a value.

```
student = {  
    "name": "Aamir",
```

</> Python



Run

Here:

```
"name" → key  
"age" → key
```



Keys must be **unique**.

✖ Duplicate key

```
student = {  
    "name": "Aamir",
```

</> Python



▶ Run

Python keeps only the last value:

```
{'name': 'John'}
```



Why?

Because one key points to one current value.

```
"name"  
  |  
  ↓  
"John"
```



The earlier "Aamir" gets replaced.

6. Accessing Dictionary Values

Use:

</> Python



▶ Run

Example:

```
student = {
```



```
"name": "Aamir",  
"age": 22  
}
```

</> Python



Run

Output:

Aamir



Dry run

Step	Operation	Result
1	Create dictionary	{"name": "Aamir", "age": 22}
2	Ask for "name"	Find key
3	Value of "name"	"Aamir"
4	print()	Aamir

7. What if the Key Does Not Exist?

Consider:

```
student = {  
    "name": "Aamir",  
    "age": 22  
}
```

</> Python



Run

There is no "city" key.

Python gives:

KeyError



Why?

Dictionary

```
|  
├── "name" → "Aamir"  
└── "age" → 22
```

Search: "city"

↓

✗ NOT FOUND

↓

KeyError

8. get() Method

A safer way to access a value is:

</> Python



▶ Run

Example:

```
student = {  
    "name": "Aamir",  
    "age": 22  
}
```

</> Python



▶ Run

Output:

None



No KeyError .

You can also give a default value:

</> Python



▶ Run

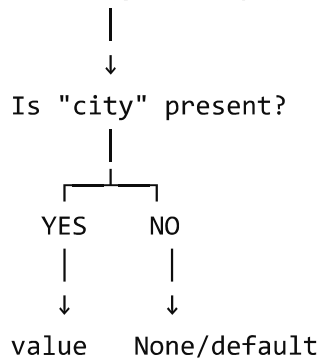
Output:

Not available



Visual

```
student.get("city")
```



Important Difference

</> Python



Run

✗ Raises `KeyError` if missing.

</> Python



Run

✓ Returns `None` if missing.

9. Adding a New Item

Simply assign a new key.

```
student = {  
    "name": "Aamir"  
}
```

```
student["age"] = 22
```

</> Python



Run

Output:

```
{'name': 'Aamir', 'age': 22}
```



Dry run

Step

Dictionary

Start

```
{"name": "Aamir"}
```

```
student["age"] = 22
```

Add "age"

Final

```
{"name": "Aamir", "age": 22}
```

Visual:

Before



```
"name" → "Aamir"
```

After

```
"name" → "Aamir"
```

```
"age" → 22
```

10. Updating a Value

If the key already exists:

```
student = {  
    "name": "Aamir",  
    "age": 22  
}
```

</> Python



Run

Now:

```
"name" → "Aamir"  
"age" → 23
```



The old 22 is replaced.

Important rule

```
</> Python
```



▶ Run

If key doesn't exist:

ADD



If key already exists:

UPDATE



Diagram

```
student["age"] = 23  
  |  
  ↓  
Does "age" exist?  
 /   \  
YES  NO  
  ↓   ↓  
UPDATE  ADD
```



11. Deleting Dictionary Items

There are several ways.

`del`

```
student = {  
    "name": "Aamir",  
    "age": 22  
}  
  
del student["age"]
```

</> Python



Run

Output:

```
{'name': 'Aamir'}
```



Visual

Before:



```
"name" → "Aamir"  
"age"  → 22      ← DELETE
```

After:

```
"name" → "Aamir"
```

12. pop()

pop() removes a key and **returns its value**.

```
student = {  
    "name": "Aamir",  
    "age": 22  
}  
  
age = student.pop("age")
```

</> Python



Run

Output:

22

`{'name': 'Aamir'}`

Difference

`</> Python`

Run

removes it.

`</> Python`

Run

removes it and gives you the removed value.

13. popitem()

Removes and returns the **last inserted key-value pair**.

```
student = {  
    "name": "Aamir",  
    "age": 22,  
    "course": "Python"  
}
```

```
item = student.popitem()
```

`</> Python`

Run

Output:

`('course', 'Python')`

Dictionary becomes:

`{`

```
"name": "Aamir".
```

</> Python



Run

14. clear()

Removes everything.

```
student = {  
    "name": "Aamir",  
    "age": 22  
}  
  
student.clear()
```

</> Python



Run

Output:

```
{}
```



Visual:

BEFORE

```
"name" → "Aamir"  
"age"  → 22
```

```
clear()  
↓
```

AFTER

```
{}
```

15. keys()

Returns dictionary keys.

```
student = {  
    "name": "Aamir",  
    "age": 22  
}
```

</> Python



Run

Conceptually:

```
name  
age
```



You can loop over them:

</> Python



Run

Output:

```
name  
age
```



16. values()

Returns all values.

```
student = {  
    "name": "Aamir",  
    "age": 22  
}
```

</> Python



Run

Output:

Aamir
22



Visual:

KEYS		VALUES
"name"	→	"Aamir"
"age"	→	22



17. items()

This is extremely important for loops.

```
student = {  
    "name": "Aamir",  
    "age": 22  
}
```

</> Python



▶ Run

Output:

name Aamir
age 22



Think of `.items()` as:

(key, value)



Visual:

```
student.items()  
├── ("name", "Aamir")  
└── ("age", 22)
```



18. Looping Through a Dictionary

Loop through keys

```
student = {  
    "name": "Aamir",  
    "age": 22  
}
```

</> Python



▶ Run

Output:

```
name  
age
```



By default, iterating over a dictionary gives its **keys**.

Loop through values

</> Python



▶ Run

Loop through both

</> Python



▶ Run

Output:

```
name = Aamir
```



`age = 22`

19. The `in` Operator

You can check whether a key exists.

```
student = {  
    "name": "Aamir",  
    "age": 22  
}
```

 Python



 Run

Output:

True



But:

 Python



 Run

is:

False



Why?

`in` checks **keys**, not values.

```
"name" → Aamir  
  ↑  
checks key  
  
"Aamir"  
  ↑  
value, not key
```



To search values:

</> Python



Run

Output:

True



20. Dictionary Length

Use:

</> Python



Run

Example:

```
student = {  
    "name": "Aamir",  
    "age": 22,  
    "course": "Python"  
}
```

</> Python



Run

Output:

3



Formula

Number of dictionary items = `len(dictionary)`

Here:

```
name    → Aamir  
age     → 22  
course  → Python
```



3 key-value pairs

21. Can Dictionary Values Have Different Types?

Yes! This is one of Python's useful features.

```
student = {  
    "name": "Aamir",      # string  
    "age": 22,            # integer  
    "marks": 95.5,        # float  
    "passed": True,        # boolean
```

</> Python



Run

A dictionary can contain different types of values.

22. Can Keys Have Different Types?

Yes, if they are valid hashable types.

Example:

```
data = {  
    "name": "Aamir",  
    101: "Student ID",
```

</> Python



Run

Common valid keys:

```
string  
integer  
float  
tuple  
boolean
```



But mutable types such as lists cannot normally be keys.

23. Why Can't a List Be a Dictionary Key?

This is connected to **hashing**.

Don't worry about the difficult word yet.

Easy meaning of "hashable"

Hashable = Python can calculate a stable value that allows the object to be used as a dictionary key.

This works:

```
data = {  
    "name": "Aamir",  
    10: "ten",  
}
```

</> Python



▶ Run

This does not:

```
data = {
```

</> Python



▶ Run

It raises:

```
TypeError: unhashable type: 'list'
```



Why?

A list can change:

```
numbers = [1, 2]
```

</> Python



▶ Run

If a changing object were used as a key, dictionary lookup could become unreliable.

24. Dictionary Mutability

A dictionary is **mutable**.

Easy meaning

Mutable = You can change it after creating it.

Example:

```
student = {  
    "name": "Aamir"  
}
```

</> Python



▶ Run

Now:

</> Python



▶ Run

You didn't create a completely new dictionary manually. You changed the existing data.

25. Nested Dictionary

A dictionary can contain another dictionary.

```
students = {  
    "student1": {  
        "name": "Aamir",  
        "age": 22  
    },  
    "student2": {  
        "name": "Rahul",  
        "age": 23  
    }  
}
```

</> Python



Run

Visual:

```
students
├── student1
│   ├── name → Aamir
│   └── age  → 22
└── student2
    ├── name → Rahul
    └── age  → 23
```



Access:

</> Python



Run

Output:

Aamir



Read it from left to right

```
students["student1"]["name"]
  |           |
```

</> Python



Run

26. Dictionary Containing a List

Very common in real programs.

```
student = {
```



```
"name": "Aamir",
```

</> Python



Run

Access:

</> Python



Run

Output:

['Python', 'SQL', 'Java']



Access one subject:

</> Python



Run

Output:

Python



Visual:

student

|
|
|— name → Aamir|
|
|— subjects

Python	SQL	Java
--------	-----	------

0

1

2

27. Copying Dictionaries

This is a common interview topic.

Consider:

```
a = {  
    "name": "Aamir"  
}
```

</> Python



▶ Run

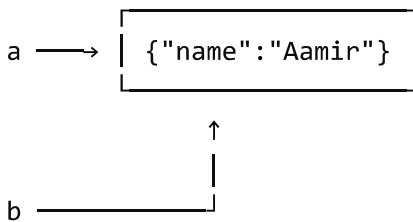
You might think:

```
a = one dictionary  
b = another dictionary
```



But that's not what happens.

Both names refer to the same dictionary.



So:

```
b["name"] = "John"
```

</> Python



▶ Run

Output:

```
{'name': 'John'}
```



🤖 a changed too.

28. copy()

To create a shallow copy:

```
a = {  
    "name": "Aamir"  
}  
  
b = a.copy()  
  
b["name"] = "John"
```

</> Python



▶ Run

Output:

```
{'name': 'Aamir'}  
{'name': 'John'}
```



Now they are separate outer dictionaries.

29. dict() Constructor

You can create dictionaries using `dict()`.

```
student = dict(name="Aamir", age=22)
```

</> Python



▶ Run

Output:

```
{'name': 'Aamir', 'age': 22}
```



You can also convert pairs:

```
data = dict(  
    ("name", "Aamir"),
```

</> Python



▶ Run

30. fromkeys()

Creates a dictionary from keys with the same initial value.

```
keys = ["name", "age", "city"]  
  
data = dict.fromkeys(keys, "unknown")
```

Python



Run

Output:

```
{  
  'name': 'unknown',  
  'age': 'unknown',  
  'city': 'unknown'  
}
```



Formula

```
dict.fromkeys(keys, value)  
      ↑      ↑  
    keys  default value
```



31. setdefault()

setdefault() is useful when a key may or may not exist.

```
student = {  
    "name": "Aamir"  
}  
  
student.setdefault("age", 22)
```

Python



Run

Output:

```
{'name': 'Aamir', 'age': 22}
```



If the key already exists:

```
student = {  
    "age": 22  
}  
  
student.setdefault("age", 30)
```

</> Python



Run

Output:

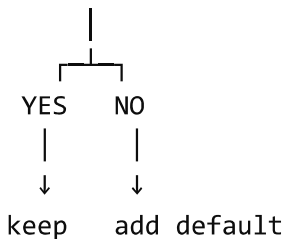
```
{'age': 22}
```



It does **not** replace the existing value.

Easy rule

Key exists?



32. update()

Used to add or update multiple values.

```
student = {  
    "name": "Aamir",  
    "age": 22
```

```
}
```

```
student.update({  
    "age": 23
```

```
</> Python
```



```
▶ Run
```

Result:

```
{  
    "name": "Aamir",  
    "age": 23,
```

```
</> Python
```



```
▶ Run
```

What happened?

```
age:  
22 → 23      UPDATE
```



```
city:  
missing → Delhi  ADD
```

33. Dictionary Comprehension

This looks difficult at first, but it is actually simple.

Normal loop

```
squares = {}  
  
for number in range(1, 6):  
    squares[number] = number * number
```

```
</> Python
```



```
▶ Run
```

Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```



Dictionary comprehension

Same thing in one line:

```
squares = {
    number: number * number
```

</> Python



Run

Formula

</> Python



Run

Easy meaning:

Create:
key : value



For every:
item

From:
iterable

34. Dictionary Comprehension Diagram

range(1, 6)

```

    |
    ↓
  1  2  3  4  5
  |  |  |  |  |
  ↓  ↓  ↓  ↓  ↓
 1²  2² 3² 4² 5²
  |  |  |  |  |
  ↓  ↓  ↓  ↓  ↓
 1   4  9 16 25

```

Dictionary:



1 → 1
2 → 4
3 → 9
4 → 16
5 → 25

35. Dictionary Comprehension With Condition

Example: Create squares only for even numbers.

```
squares = {  
    n: n * n  
    for n in range(1, 11)  
    if n % 2 == 0  
}
```

</> Python



▶ Run

Output:

```
{2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```



Formula

</> Python



▶ Run

36. Counting With Dictionaries

This is one of the most important practical uses.

Suppose:

</> Python



▶ Run

We want:

b → 1
a → 3
n → 2



Code:

```
text = "banana"

count = {}

for char in text:
    if char in count:
        count[char] += 1
    else:
        count[char] = 1
```

</> Python



Run

Output:

{'b': 1, 'a': 3, 'n': 2}



Dry run

Character	Is it already present?	Dictionary
b	No	{'b': 1}
a	No	{'b': 1, 'a': 1}
n	No	{'b': 1, 'a': 1, 'n': 1}
a	Yes	{'b': 1, 'a': 2, 'n': 1}
n	Yes	{'b': 1, 'a': 2, 'n': 2}
a	Yes	{'b': 1, 'a': 3, 'n': 2}

Concept

Character



Already seen?

/

\

NO

YES



1

increase by 1



37. get() Makes Counting Easier

The previous code can be shortened:

```
text = "banana"

count = {}

for char in text:
    count[char] = count.get(char, 0) + 1
```

</> Python



Run

Output:

```
{'b': 1, 'a': 3, 'n': 2}
```



Understand this line

</> Python



Run

Upgrade

means.



"Give me the current count. If the character doesn't exist, give me 0."

Then:

</> Python



Run

```
+ 1
```

increases it.

38. defaultdict

Python also has `collections.defaultdict`.

</> Python



▶ Run

```
from collections import defaultdict

count = defaultdict(int)

for char in "banana":
    count[char] += 1

print(count)
```

Conceptually:

Missing key → automatically starts with 0



This is especially useful for grouping and counting.

39. Grouping Data Using Dictionaries

Suppose:

</> Python



▶ Run

```
students = [
    ("Aamir", "Python"),
    ("Rahul", "Java"),
    ("John", "Python"),
    ("Sara", "Java")
]
```

We want:

Python → Aamir, John
Java → Rahul, Sara



Code:

</> Python



▶ Run

```
groups = {}

for name, course in students:
    groups.setdefault(course, []).append(name)

print(groups)
```

Result:

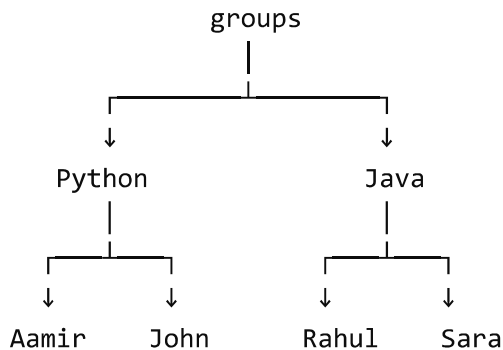
</> Python



▶ Run

```
{
    "Python": ["Aamir", "John"],
    "Java": ["Rahul", "Sara"]
}
```

Diagram



This pattern is extremely useful in programming interviews.

40. Dictionary Performance — Why Dictionaries Are Fast

This is an important advanced concept.

A dictionary is implemented using a **hash table**.

Easy explanation

Imagine a huge library.

Instead of checking every book:

```
Book 1
Book 2
Book 3
Book 4
...
Book 1,000,000
```



you use a system that quickly tells you where the book should be.

Dictionary hashing works similarly.

```
Key
|
↓
Hash function
|
↓
Location
|
↓
Value
```



Example:

```
"name"
|
↓
hash()
|
↓
some internal location
|
↓
"Aamir"
```



41. Hash Table Visualization

KEY





You don't normally calculate this yourself. Python handles it.

42. Dictionary Time Complexity

For a normal dictionary operation, average-case complexity is approximately:

Operation	Average Time
<code>d[key]</code>	$O(1)$
<code>d[key] = value</code>	$O(1)$
<code>key in d</code>	$O(1)$
<code>d.get(key)</code>	$O(1)$
<code>del d[key]</code>	$O(1)$
Loop through dictionary	$O(n)$

What does $O(1)$ mean?

Very roughly:

The operation usually takes about the same amount of lookup work even when the dictionary becomes much larger.

What does $O(n)$ mean?

Work grows with the number of items.

If:

```
n = 10
```



you may inspect about 10 items during a full traversal.

If:

```
n = 1,000,000
```



a full traversal may involve about one million items.

⚠️ These are complexity models, not exact running times.

43. Dictionary Ordering

Modern Python dictionaries preserve **insertion order**.

Example:

</> Python



▶ Run

```
student = {}

student["name"] = "Aamir"
student["age"] = 22
student["city"] = "Delhi"

print(student)
```

The order is:

```
name
age
city
```



because that was the insertion order.

Important distinction

Dictionary



Preserves insertion order

But dictionary order is **not the same thing as sorting**.

Example:

</> Python



Run

```
{  
    "z": 1,  
    "a": 2  
}
```

The dictionary does not automatically become:

</> Python



Run

```
{  
    "a": 2,  
    "z": 1  
}
```

44. Sorting a Dictionary

You can sort its keys:

</> Python



Run

```
scores = {  
    "John": 80,  
    "Aamir": 95,  
    "Rahul": 85  
}  
  
sorted_scores = dict(sorted(scores.items()))  
  
print(sorted_scores)
```

Result:

</> Python



Run

```
{  
    "Aamir": 95,  
    "Rahul": 85,  
    "John": 80  
}
```

```
"John": 80,  
"Rahul": 85  
}
```

Here:

</> Python



▶ Run

```
scores.items()
```

produces key-value pairs, and `sorted()` sorts them by key by default.

45. Sorting by Value

Very useful!

</> Python



▶ Run

```
scores = {  
    "John": 80,  
    "Aamir": 95,  
    "Rahul": 85  
}  
  
result = dict(  
    sorted(  
        scores.items(),  
        key=lambda item: item[1]  
    )  
)  
  
print(result)
```

Output:

```
{'John': 80, 'Rahul': 85, 'Aamir': 95}
```



What does this mean?

</> Python



▶ Run

```
key=lambda item: item[1]
```

means:

Sort using the **value** part of each (key, value) pair.

Remember:

```
item = ("John", 80)
```

```
item[0] → "John"
```

```
item[1] → 80
```



46. Finding the Maximum Value

Example:

 Python



 Run

```
scores = {  
    "John": 80,  
    "Aamir": 95,  
    "Rahul": 85  
}  
  
highest = max(scores, key=scores.get)  
  
print(highest)
```

Output:

Aamir



Why?

Python checks:

```
John → 80
```

```
Aamir → 95
```

```
Rahul → 85
```



Highest value:

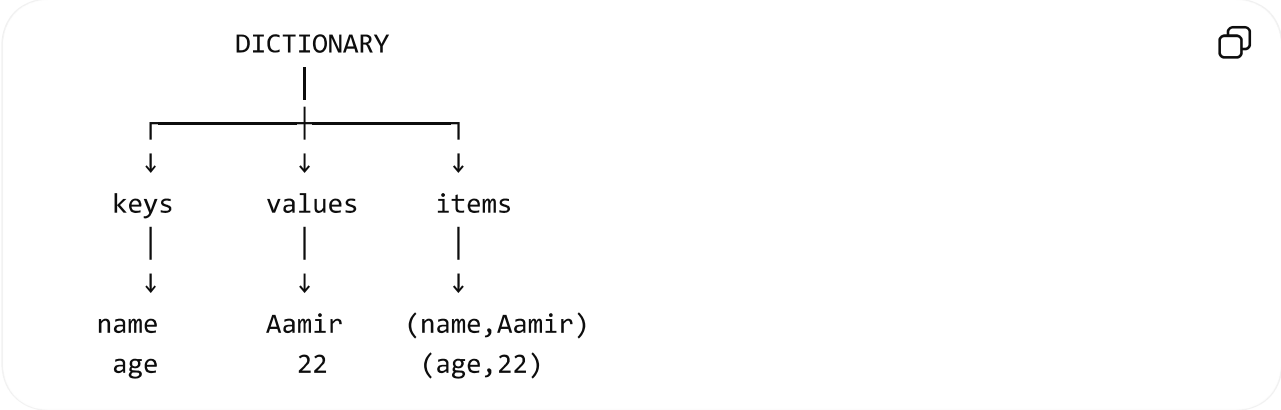
95



Key connected to 95:

Aamir

47. keys , values , and items — Master Diagram



Remember:

</> Python

```
d.keys()    # keys
d.values()  # values
d.items()   # key + value
```

Run

48. Dictionary Methods Cheat Sheet

Method	Easy meaning
get()	Get value safely
keys()	Get keys
values()	Get values
items()	Get key-value pairs
update()	Add/update multiple items

Method	Easy meaning
<code>pop()</code>	Remove specific key and return value
<code>popitem()</code>	Remove last inserted pair
<code>clear()</code>	Remove everything
<code>copy()</code>	Make shallow copy
<code>setdefault()</code>	Get key or insert default
<code>fromkeys()</code>	Create dictionary from keys

49. Common Mistakes

Mistake 1: Using square brackets to create dictionary

Wrong:

 Python



 Run

```
student = [  
    "name": "Aamir"  
]
```

Correct:

 Python



 Run

```
student = {  
    "name": "Aamir"  
}
```

Mistake 2: Forgetting the colon

Wrong:

 Python



 Run

```
student = {  
    "name", "Aamir"  
}
```

Correct:

</> Python



▶ Run

```
student = {  
    "name": "Aamir"  
}
```

✗ Mistake 3: Assuming dictionary uses numeric indexes

Wrong thinking:

</> Python



▶ Run

```
student[0]
```

A dictionary normally uses keys:

</> Python



▶ Run

```
student["name"]
```

✗ Mistake 4: Accessing a missing key

</> Python



▶ Run

```
student["city"]
```

can produce:

KeyError



Safer:

</> Python



▶ Run

```
student.get("city")
```

✖ Mistake 5: Duplicate keys

</> Python



▶ Run

```
data = {  
    "age": 20,  
    "age": 30  
}
```

Final value:

30



The second assignment replaces the first.

✖ Mistake 6: Using a list as a key

</> Python



▶ Run

```
data = {  
    [1, 2]: "hello"  
}
```

✖ TypeError

Use a tuple if appropriate:

</> Python



▶ Run

```
data = {  
    (1, 2): "hello"  
}
```

✖ Mistake 7: Changing dictionary size while iterating

Be careful with code like:

</> Python



Run

```
for key in data:  
    del data[key]
```

This can raise:

RuntimeError



A safer approach can be:

</> Python



Run

```
for key in list(data):  
    del data[key]
```

50. Complete Practical Example

Let's build a student record.

</> Python



Run

```
student = {  
    "name": "Aamir",  
    "age": 22,  
    "course": "Python",  
    "marks": 95  
}  
  
# Read  
print(student["name"])  
  
# Update  
student["marks"] = 98  
  
# Add  
student["city"] = "Delhi"  
  
# Check  
if "course" in student:  
    print("Course exists")  
  
# Loop  
for key, value in student.items():
```

```
print(key, ":", value)

# Delete
student.pop("age")

print(student)
```

Dry run

Step	Operation	Result
1	Create dictionary	4 items
2	Read name	Aamir
3	Change marks	95 → 98
4	Add city	"city" → "Delhi"
5	Check course	True
6	Loop	Print all pairs
7	Remove age	Age deleted

Final dictionary:

</> Python

Copy

Run

```
{
  "name": "Aamir",
  "course": "Python",
  "marks": 98,
  "city": "Delhi"
}
```

51. Real-World Example: Phone Contacts

Imagine your phone contacts.

</> Python

Copy

Run

```
contacts = {
```

```
"Aamir": "9876543210",  
"Rahul": "9876543211",  
"John": "9876543212"  
}
```

Lookup:

</> Python



▶ Run

```
print(contacts["Aamir"])
```

Output:

9876543210



This is a perfect dictionary use case:

Person's name → Phone number



52. Real-World Example: Product Prices

</> Python



▶ Run

```
prices = {  
    "apple": 100,  
    "banana": 40,  
    "milk": 60  
}
```

Find price:

</> Python



▶ Run

```
print(prices["apple"])
```

Output:

100



Update:

</> Python



▶ Run

```
prices["apple"] = 110
```

Add:

</> Python



▶ Run

```
prices["bread"] = 50
```

53. Real-World Example: API/JSON Data

Dictionaries are extremely common when working with JSON and APIs.

Example:

</> Python



▶ Run

```
user = {  
    "id": 101,  
    "name": "Aamir",  
    "email": "aamir@example.com",  
    "active": True  
}
```

You might access:

</> Python



▶ Run

```
print(user["email"])
```

Output:

```
aamir@example.com
```



This is why dictionaries are extremely important in:

- Web development
- APIs
- Data science
- Automation
- Backend programming

- Configuration files
- JSON processing

54. Dictionary vs Set

These are often confused because both use `{}` .

Dictionary

 Python



 Run

```
student = {  
    "name": "Aamir"  
}
```

Contains:

key : value



Set

 Python



 Run

```
numbers = {1, 2, 3}
```

Contains:

values only



Empty structures

This is important:

 Python



 Run

```
{}
```

means:

Empty dictionary

An empty set is:

</> Python

set()

Run

55. Dictionary vs List vs Tuple vs Set

Feature	List	Tuple	Set	Dictionary
Syntax	[]	()	{ }	{key:value}
Ordered	Yes	Yes	No guaranteed indexing	Insertion order
Mutable	Yes	No	Yes	Yes
Duplicates	Yes	Yes	No	Keys: No
Access	Index	Index	Membership	Key
Key-value pairs	No	No	No	Yes

Easy memory

LIST → positions

TUPLE → fixed collection

SET → unique values

DICTIONARY → key → value

Run

56. Advanced: Dictionary Unpacking

You can use `**` .

</> Python

student = {
 "name": "Aamir",
 "age": 22
}

Run

```
}

new_student = {
    **student,
    "course": "Python"
}

print(new_student)
```

Output:

</> Python



Run

```
{
    "name": "Aamir",
    "age": 22,
    "course": "Python"
}
```

Meaning

****student**

↓

Take all key-value pairs from student
and put them here.



57. Combining Dictionaries

Modern Python supports:

</> Python



Run

```
a = {"name": "Aamir"}
b = {"age": 22}

c = a | b

print(c)
```

Output:

```
{'name': 'Aamir', 'age': 22}
```



You can also update with:

</> Python



▶ Run

```
a |= b
```

58. Nested Data — A Very Important Pattern

Real applications often contain structures like:

</> Python



▶ Run

```
company = {  
    "name": "ABC",  
    "employees": [  
        {  
            "name": "Aamir",  
            "role": "Developer"  
        },  
        {  
            "name": "Rahul",  
            "role": "Designer"  
        }  
    ]  
}
```

Visual:

company
├── name → ABC
└── employees
 │
 ↓

List
0 → Dictionary
name → Aamir
role → Dev
1 → Dictionary
name → Rahul

role → Design

Access Aamir's role:

</> Python



▶ Run

```
print(company["employees"][0]["role"])
```

Output:

Developer



Read it carefully:

```
company
  ↓
employees
  ↓
index 0
  ↓
role
```



59. Master Mental Model

Whenever you see:

</> Python



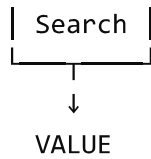
▶ Run

```
d[key]
```

think:

```
d
|
↓
DICTIONARY
|
↓
key
|
↓
```





Example:

</> Python



▶ Run

```
student["age"]
```

Think:

```
student
  ↓
find "age"
  ↓
22
```



60. Dictionary Formula Sheet

Access

</> Python



▶ Run

```
d[key]
```

Parameters:

- d = dictionary
- key = key you want to search

Safe access

</> Python



▶ Run

```
d.get(key, default)
```

Parameters:

- d = dictionary

- `key` = key to search
- `default` = value returned if key doesn't exist

Example:

</> Python



▶ Run

```
student.get("city", "Unknown")
```

Length

</> Python



▶ Run

```
len(d)
```

- `d` = dictionary

Returns:

Number of key-value pairs.

Update

</> Python



▶ Run

```
d.update(other)
```

- `d` = dictionary being changed
- `other` = dictionary/mapping containing new values

Remove

</> Python



▶ Run

```
d.pop(key)
```

- `d` = dictionary
- `key` = key to remove

Dictionary comprehension

</> Python



▶ Run

```
{key: value for item in iterable}
```

- `key` = expression used as dictionary key
- `value` = expression used as dictionary value
- `item` = current element
- `iterable` = collection being processed

61. Interview Questions

Q1. What is a dictionary?

Answer:

A dictionary is a mutable Python data structure that stores data as **key-value pairs**.

Example:

</> Python



▶ Run

```
student = {  
    "name": "Aamir",  
    "age": 22  
}
```

Q2. Are dictionary keys unique?

Yes.

</> Python



▶ Run

```
d = {  
    "a": 10,  
    "a": 20  
}
```

The final value for "a" is:

20



Q3. Are dictionary values required to be unique?

No.

</> Python



Run

```
d = {  
    "a": 10,  
    "b": 10  
}
```

This is completely valid.

Both keys have the same value.

Q4. Can a list be a dictionary key?

No.

</> Python



Run

```
{[1, 2]: "hello"}
```

raises a `TypeError` because a list is unhashable.

Q5. Can a tuple be a dictionary key?

Yes, if the tuple's contents are hashable.

</> Python



Run

```
d = {  
    (1, 2): "point"  
}
```

Q6. Difference between `d[key]` and `d.get(key)` ?

</> Python



▶ Run

```
d[key]
```

raises `KeyError` if the key is missing.

</> Python



▶ Run

```
d.get(key)
```

returns `None` by default if the key is missing.

Q7. How do you check whether a key exists?

</> Python



▶ Run

```
if "name" in student:  
    print("Exists")
```

Q8. How do you loop through both keys and values?

</> Python



▶ Run

```
for key, value in student.items():  
    print(key, value)
```

Q9. How do you remove a dictionary item?

Several options:

</> Python



▶ Run

```
del d[key]
```

or:

</> Python



▶ Run

```
d.pop(key)
```

or:

</> Python



▶ Run

```
d.popitem()
```

or remove everything:

</> Python



▶ Run

```
d.clear()
```

Q10. Why are dictionaries generally fast for lookup?

Because Python dictionaries use a **hash-table-based implementation**, allowing average-case constant-time lookup.

```
key  
↓  
hash  
↓  
location  
↓  
value
```



62. Interview Dry-Run Question

What is the output?

</> Python



▶ Run

```
d = {  
    "a": 10,  
    "b": 20
```

```
}  
  
d["a"] = 100  
d["c"] = 30  
  
print(d)
```

Dry run

Start:

```
a → 10  
b → 20
```



After:

</> Python



▶ Run

```
d["a"] = 100
```

we have:

```
a → 100  
b → 20
```



After:

</> Python



▶ Run

```
d["c"] = 30
```

we have:

```
a → 100  
b → 20  
c → 30
```



Answer:

</> Python



▶ Run

```
{'a': 100, 'b': 20, 'c': 30}
```

63. Interview Dry-Run Question

What is the output?

 Python



 Run

```
d = {  
    "a": 1,  
    "b": 2,  
    "c": 3  
}  
  
for key, value in d.items():  
    print(key, value)
```

Answer:

```
a 1  
b 2  
c 3
```



Because `.items()` gives:

```
(key, value)
```



64. Interview Dry-Run Question

What is the output?

 Python

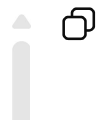


 Run

```
d = {  
    "name": "Aamir",  
    "age": 22  
}  
  
print(d.get("city", "Unknown"))
```

Dry run:

```
Search "city"  
↓
```



Not found
↓
Use default
↓
"Unknown"

Answer:

Unknown



65. Interview Coding Question — Count Frequencies

Problem

Given:

</> Python



▶ Run

```
numbers = [1, 2, 2, 3, 1, 2]
```

count each number.

Solution

</> Python



▶ Run

```
numbers = [1, 2, 2, 3, 1, 2]

frequency = {}

for number in numbers:
    frequency[number] = frequency.get(number, 0) + 1

print(frequency)
```

Output:

```
{1: 2, 2: 3, 3: 1}
```



Dry run

Number	Previous count	New count
1	0	1
2	0	1
2	1	2
3	0	1
1	1	2
2	2	3

Final:

1 → 2
2 → 3
3 → 1





66. Problems You Should Practice

Start with these in order:



Beginner

1. Create a dictionary of student information.
2. Read a value using a key.
3. Add a new key.
4. Update an existing key.
5. Delete a key.
6. Check whether a key exists.
7. Print all keys.
8. Print all values.
9. Print all key-value pairs.
10. Find dictionary length.



Intermediate

11. Count characters in a string.

12. Count numbers in a list.
13. Find the student with the highest marks.
14. Find the student with the lowest marks.
15. Group students by course.
16. Merge two dictionaries.
17. Sort a dictionary by key.
18. Sort a dictionary by value.
19. Create a dictionary using comprehension.
20. Filter dictionary items.

● Advanced

21. Work with nested dictionaries.
22. Process JSON-like data.
23. Group complex records.
24. Find duplicate values.
25. Build a frequency counter.
26. Implement a simple cache.
27. Solve two-sum using a dictionary.
28. Find the first repeated element.
29. Find the first non-repeated character.
30. Use dictionaries for graph adjacency lists.



67. Most Important Pattern: Two Sum

A famous interview problem.

Given:

</> Python



▶ Run

```
numbers = [2, 7, 11, 15]
target = 9
```

Find two numbers whose sum is 9 .

Dictionary solution:

</> Python



▶ Run

```
numbers = [2, 7, 11, 15]
target = 9

seen = {}

for i, number in enumerate(numbers):
    needed = target - number

    if needed in seen:
        print(seen[needed], i)
        break

    seen[number] = i
```

Output:

0 1



Because:

```
numbers[0] = 2
numbers[1] = 7

2 + 7 = 9
```



Core idea

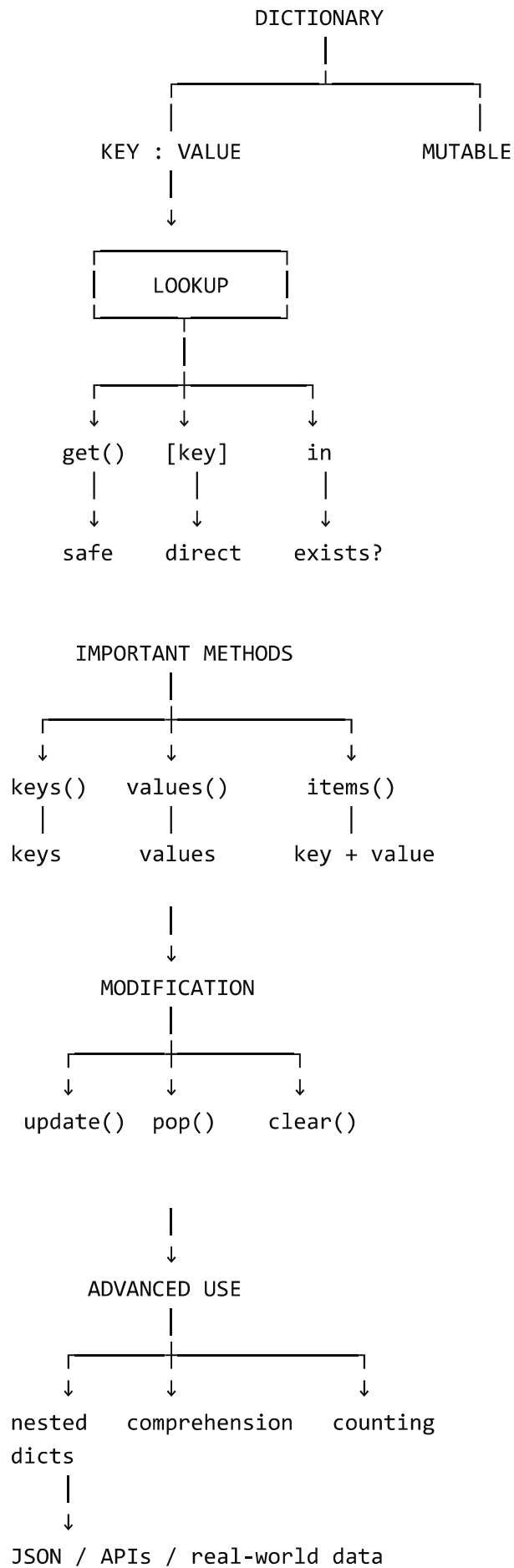
```
Current number
  ↓
target - current
  ↓
needed number
  ↓
Have we already seen it?
  ↓
YES
  ↓
Found answer
```



This is a major example of why dictionary lookup being approximately $O(1)$ is useful.



68. Final One-Page Memory Map



Final Color Notes — Remember These

-  Dictionary = Key → Value
-  Keys must be unique.
-  Values can repeat.
-  Dictionary is mutable.
-  **Missing** `d[key]` → `KeyError` .
-  **Missing** `d.get(key)` → `None` **by default.**
-  `d.keys()` → **keys**
-  `d.values()` → **values**
-  `d.items()` → **key-value pairs**
-  `d[key] = value` → **add OR update**
-  `del d[key]` → **delete**
-  `d.pop(key)` → **delete + return value**
-  `d.clear()` → **remove everything**
-  `key in d` → **checks keys**
-  Dictionary lookup is approximately $O(1)$ on average.
-  Lists are not valid dictionary keys because they are unhashable.
-  Dictionary comprehension:

 Python

```
{key: value for item in iterable}
```

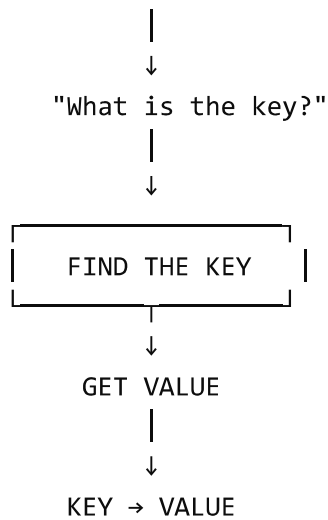


 Run

★ The single most important mental model:

DICTIONARY





If you understand `key -> value`, **lookup**, **add/update**, **delete**, **looping**, `get()`, **nested dictionaries**, **dictionary comprehension**, **counting**, and the basic idea of **hash tables**, you have the core of Python dictionaries from beginner through interview level.



Is this conversation helpful so far?



ChatGPT can make mistakes. Check important info.